

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	June 2026
Project Name	AI-Enhanced Intrusion Detection System
Maximum Marks	As Applicable

Technical Architecture:

The AI-Enhanced IDS follows a 3-tier architecture: a frontend dashboard tier, a Flask REST API middle tier, and an ML model + data tier. All components run locally in a Python virtual environment (Python 3.11.9) on Windows with VS Code.

Architecture Flow:

Network Traffic Data (NSL-KDD CSV) → Pandas Preprocessing → Scikit-learn Random Forest → Joblib Model (.pkl) → Flask REST API → JavaScript Dashboard (Chart.js) → SOC Analyst

Table 1: Components & Technologies

S.No.	Component	Description	Technology
1.	User Interface	Real-time SOC dashboard for security analysts	HTML5, CSS3, JavaScript
2.	Application Logic - Frontend	Dashboard data polling, chart rendering, alert display	Vanilla JS, Fetch API, Chart.js 4.x
3.	Application Logic - Backend	REST API serving ML predictions	Python 3.11, Flask 3.x, Flask-CORS
4.	Machine Learning Model	5-class network traffic classifier	Scikit-learn RandomForestClassifier
5.	Data Preprocessing	Feature encoding and scaling pipeline	Pandas, LabelEncoder, StandardScaler
6.	Model Serialization	Persist trained model to disk	Joblib (.pkl format)
7.	Dataset	Benchmark IDS training/testing dataset	NSL-KDD (CSV, 41 features)
8.	Infrastructure (Server)	Development and deployment environment	Python venv, Flask dev server, Unicorn

Table 2: Application Characteristics

S.No.	Characteristics	Description	Technology
1.	Open-Source Frameworks	All frameworks are open-source	Flask, Scikit-learn, Pandas, Chart.js

2.	Security Controls	Input validation on API endpoints, CORS policy	Flask-CORS, input sanitization
3.	Scalable Architecture	3-tier: Frontend, API, ML model	Stateless REST API, replaceable ML backend
4.	Availability	Runs on local server; cloud-deployable	Flask/Gunicorn on local or cloud VM
5.	Performance	API response <200ms, dashboard refresh <=5 sec	In-memory model, polling with setInterval